

实验三

年级： 2022 专业： 计算机科学与技术

姓名： 姚知言 学号： 2211290

1 实验内容

(1) 用栈实现队列，并实现操作：队尾入队 `enqueue()`，查看队首队尾 `peek_front()`、`peek_back()`，弹出队首队尾 `dequeue_front()`、`dequeue_back()`，判断队列是否为空 `is_empty()`；

(2) 单调递增队列指的是队列内元素按照非递减的次序排列的队列，队首的元素为最小值。请用单调递增队列解决以下问题：给定一个整数数组 `nums` 和一个固定大小 `k`，从数组的最左侧开始，每次向右移动一位，依次输出当前位置及其右侧的 `k` 个元素中的最小值。

说明：栈的实现可以用上次实验的结果或者直接调用栈的函数

2 设计思想

该程序的设计思想如下：

根据题目要求，创建 `myqueue` 类，调用 `stack` 库，用栈完成队列的定义，并按照题意完成队尾入队 `enqueue()`，查看队首队尾 `peek_front()`、`peek_back()`，弹出队首队尾 `dequeue_front()`、`dequeue_back()`，判断队列是否为空 `is_empty()` 的操作，并完成 `queue_size()` 判断队列内元素数量，第二问使用。

由于栈先进后出和队列先进先出的特性，通过定义临时栈将原栈倒序的方式可以完成队首的查看和弹出。

对于 (1)，通过一个 `double` 数列检查函数功能，先查看队首队尾，然后弹

出队首队尾，然后通过检查队列是否为空依次将队首元素弹出，从而实现检查以上所有函数。

对于(2),定义2个新队列 `sorted_queue`, `num_que`,将数组存入 `sorted_queue` 队列,并将元素所在位置存入 `num_que` 队列中,在每个元素 `x`,对于单调递增的要求,将队列从后往前遍历,将队列内所有小于 `x` 的元素删除后插入 `x` (`dequeue_back()`通过调用 `dequeue_front()`实现),从而保证队列为单调递增队列。并同时检查队列长度,将 `num_que` 中对应元素同时删除。

从第 `k` 个元素开始,需要先通过检查 `num_que` 将队列超出范围的数据从队列头弹出(包括 `sorted_queue` 和 `num_que` 中),然后进行插入操作(如上段所述),并且根据题意打印队列头的元素。

在输入端,通过 `getchar` 将关键信息截取存入数组中。

3 程序效果

程序的运行效果,输入及输出的相关要求和具体执行结果如下所示:

3.1 输入

(1) in: 3.5,4.5,5.5,6.5,7.5

(2) in: nums=[4,-2,1,6,-5,8,9,3],k=3

3.2 输出

(1) out: back:7.5

front:3.5

dequeue front:3.5

dequeue back:7.5

dequeue front:4.5

dequeue front:5.5

dequeue front:6.5

empty.

(2) out: [-2,-2,-5,-5,-5,3]

4 核心代码

```
#include <iostream>

#include <stack>

using namespace std;

template<class T>

class myqueue{

public:

    myqueue();

    ~myqueue() {delete que;}

    void enqueue(T x);

    void sorted_enqueue(T x);

    T dequeue_front();

    T peek_front();

    T peek_back() {return que->top();}

    T dequeue_back();

    int queue_size() {return que->size();}

    bool is_empty() {return que->empty();}

private:

    stack<T>* que;

};

template<class T>

myqueue<T>::myqueue() {

    que=new stack<T>;

}

template<class T>

void myqueue<T>::enqueue(T x) {

    que->push(x);

}

template<class T>
```

```

void myqueue<T>::sorted_enqueue(T x) {
    while((!is_empty())&& x < peek_back()) {
        dequeue_back();
    }
    enqueue(x);
}

```

```

template<class T>
T myqueue<T>::dequeue_front() {
    auto* temp = new stack<T>;
    T t1, t;
    while(!que->empty()) {
        t = que->top();
        que->pop();
        temp->push(t);
    }
    t = temp->top();
    temp->pop();
    while(!temp->empty()) {
        t1 = temp->top();
        temp->pop();
        que->push(t1);
    }
    delete temp;
    return t;
}

```

```

template<class T>
T myqueue<T>::peek_front() {
    auto* temp = new stack<T>;
    T t1, t;

```

```

while(!que->empty()) {
    t=que->top();
    que->pop();
    temp->push(t);
}

t=temp->top();
while(!temp->empty()) {
    t1=temp->top();
    temp->pop();
    que->push(t1);
}

delete temp;

return t;
}

template <class T>
T myqueue<T>::dequeue_back() {
    T t, t1;
    auto* temp=new myqueue<T>;
    while(!que->empty()) {
        t1=dequeue_front();
        temp->enqueue(t1);
    }

    t=temp->peek_back();
    while(!temp->is_empty()) {
        t1=temp->dequeue_front();
        if(!temp->is_empty())
            que->push(t1);
    }

    return t;
}

```

```

}

void op1() {
    double n;
    myqueue<double>ex;
    do {
        scanf("%lf", &n);
        ex.enqueue(n);
    }
    while (getchar()!=' ');
    cout<<"back:"<<ex.peek_back()<<"\nfront:"<<ex.peek_front();
    cout<<"\ndequeue          front:"<<ex.dequeue_front()<<"\ndequeue
back:"<<ex.dequeue_back();
    while(!ex.is_empty()) {
        cout<<"\ndequeue front:"<<ex.dequeue_front();
    }
    cout<<"\nempty. \n";
}

void op2() {
    int num[10000], size=0, k;
    while (getchar()!=' ');
    do scanf("%d", &num[size++]);
    while(getchar()!=' ');
    while (getchar()!=' ');
    scanf("%d", &k);
    myqueue<int> sorted_queue, num_que;
    for(int a=0; a<k-1; a++) {
        int t=sorted_queue.queue_size();
        sorted_queue.sorted_enqueue(num[a]);
        while(t+1>sorted_queue.queue_size()) {

```

```

        num_que.dequeue_back();

        t--;
    }

    num_que.sorted_enqueue(a);
}

printf("[");

for(int a=k-1;a<size;a++){
    if(a-num_que.peek_front()>=k){
        sorted_queue.dequeue_front();
        num_que.dequeue_front();
    }

    int t=sorted_queue.queue_size();
    sorted_queue.sorted_enqueue(num[a]);
    while(t+1>sorted_queue.queue_size()){
        num_que.dequeue_back();
        t--;
    }

    num_que.sorted_enqueue(a);
    printf("%d", sorted_queue.peek_front());
    if(a<size-1)printf(", ");
}

printf("]");
}

int main() {
    op1();
    op2();
    return 0;
}

```

5 总结

在本次实验中，复习了 c++ 的基本语法，对栈和队列，单调递增队列的了解有所加深，对栈和队列的性质得到了进一步了解，并了解了用栈实现队列的方法和单调递增队列查找区间最小值的方法。